Introduction to Bioinformatics

Multiple Alignment

Somayyeh Koohi

Department of Computer Engineering

Sharif University of Technology

Fall 2025

Adapted with modifications from lecture notes prepared by Phillip Compeau,

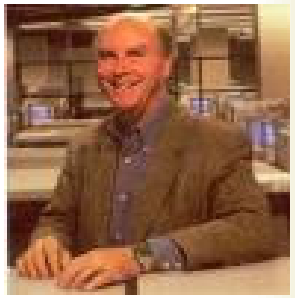
Bioinformatics Algorithms: An Active LearningApproach

Outline

- Multiple Sequence Alignment (MSA)
- Generalize DP to 3 sequence alignment
- Heuristic approaches to MSA
  I. Greedy alignment
  II. Star alignments
  III. Progressive alignment – ClustalW

Multiple Alignment versus Pairwise Alignment

- Up until now we have only tried to align two sequences.
- What about more than two? And what for?
- A faint similarity between two sequences becomes significant if present in many.
- Multiple sequence alignment can reveal subtle similarities that pairwise alignment does not reveal.

Why MSA?

- If sequence similarity is weak, pairwise alignment may not identify biologically related sequences.
- Bioinformaticians sometimes say that while pairwise alignment whispers, multiple alignment shouts.

Why MSA?

- In order to reveal the relationship between a group of sequences (homology)
- **Simultaneous alignment** of similar gene sequences may
  - Discover the conserved regions in genes
  - Determine the consensus sequence of these aligned sequences
- Help defines a protein family that may share a common biochemical function or evolutionary origin and thus reveals an evolutionary history of the sequences.
- Help prediction of the secondary and tertiary structures of new sequences

MSA Applications

- MSA is central to many bioinformatics applications
  - Phylogenetic tree
  - Motifs
  - Patterns
  - Structure prediction (RNA, protein)

What is MSA

- Indicates relationship between residues of different sequences
- Reveals similarity/disimilarity

Multiple Alignment Problem: *Find the highest-scoring alignment between multiple strings.*

- **Input:** A collection of *t* strings.
- **Output:** A multiple alignment of these strings having maximal score.

Dynamic Programming

- Dynamic Programming allow Optimal Alignment between two sequences
- Allow Insertion and Deletion or Alignment with gaps
- Needlman and Wunsh Algorithm (1970) for global alignment
- Smith & Waterman Algorithm (1981) for local alignment

From pairwise to multiple alignment

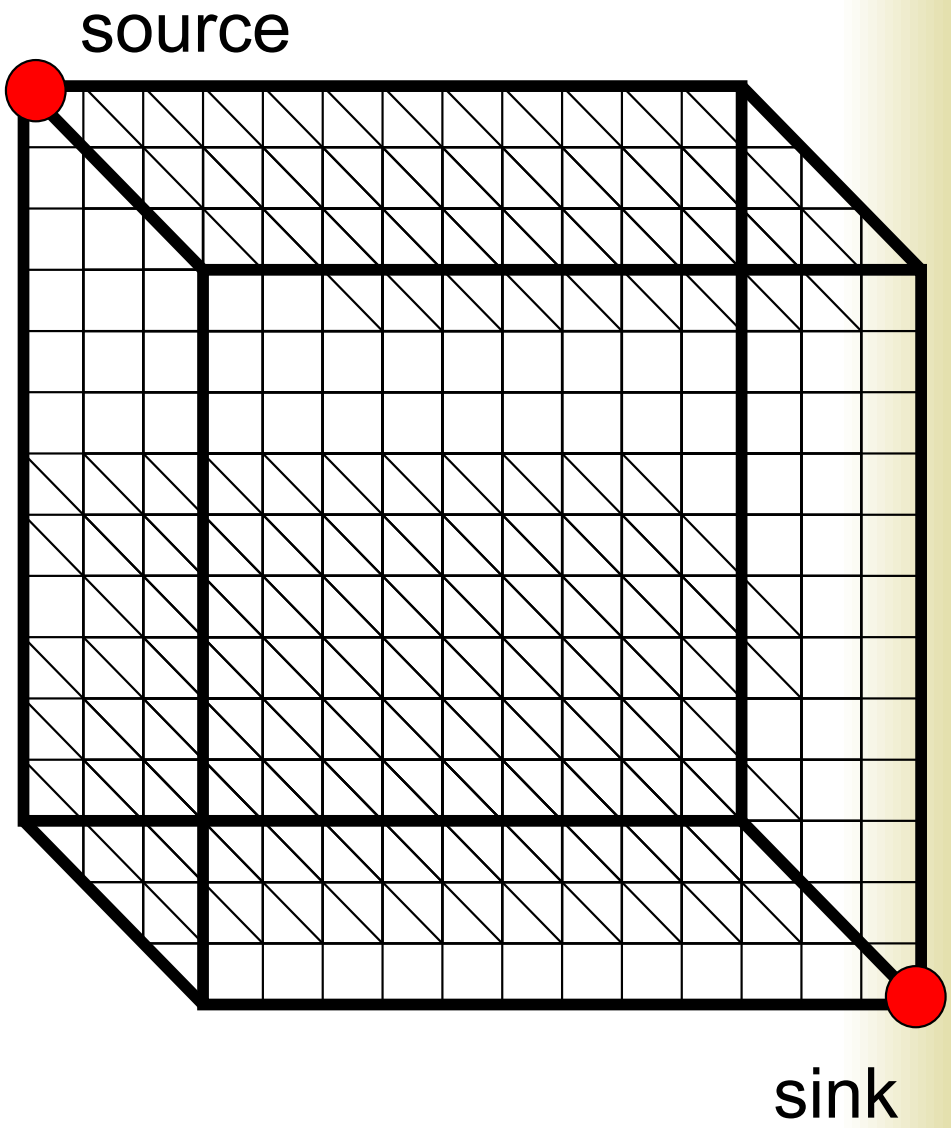
- Alignment of 2 sequences is represented as a 2-row matrix
- In a similar way, we represent alignment of 3 sequences as a 3-row matrix

```
A T _ G C G _
A _ C G T _ A
A T C A C _ A
```

- Score: more conserved columns, better alignment

Aligning three sequences

- Same strategy as aligning two sequences
- Use a 3-D "Manhattan Cube", with each axis representing a sequence to align
- For global alignments, go from source to sink



Alignment Paths

0	1	1	2	3	4
	A	--	T	G	C
0	1	2	3	3	4
	A	A	T	--	C
0	0	1	2	3	4
	--	A	T	G	C

x coordinate

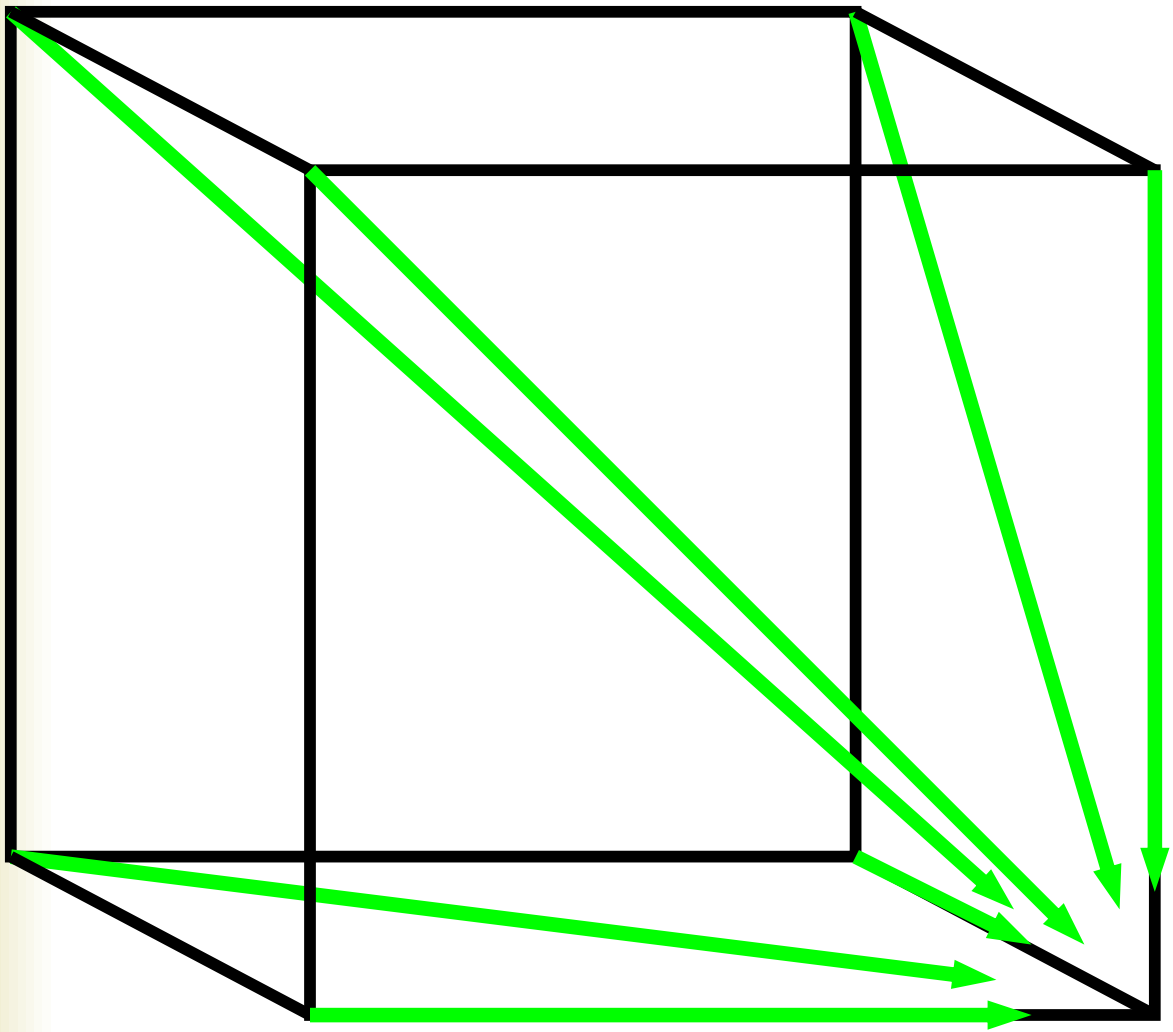
y coordinate

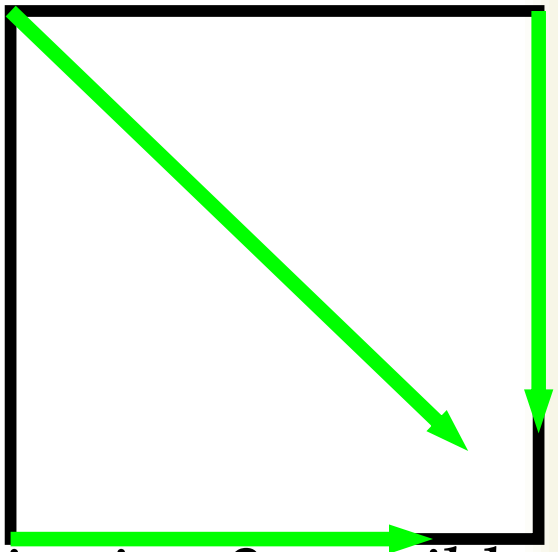
z coordinate

- Resulting path in *(x,y,z)* space:

$(0,0,0) \rightarrow (1,1,0) \rightarrow (1,2,1) \rightarrow (2,3,2) \rightarrow (3,3,3) \rightarrow (4,4,4)$

DP recursion (3 edges vs 7)

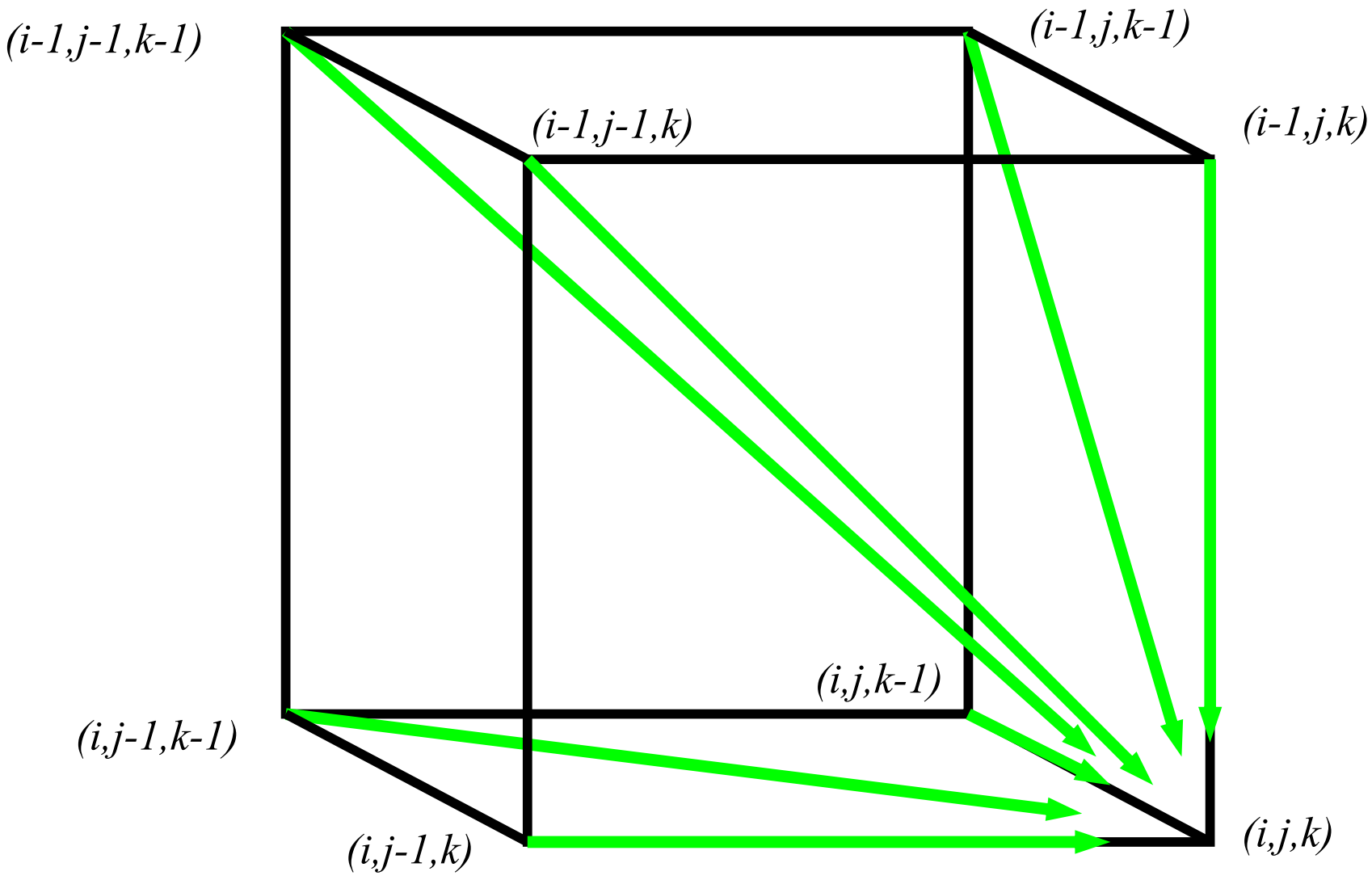




Pairwise: 3 possible paths (match/mismatch, insertion, and deletion)

In **3-D**, 7 edges in each unit cube

Architecture of 3D alignment cell



DP Alignment Examples

- All three match/mismatch

AAG
AGT
AAA

- Sequence 1 & 2 match/mismatch with gap in 3

AAG
AGT
AA-

- Sequence 1 & 3 match/mismatch with gap in 2

AAG
AG-
AAA

- Sequence 2 & 3 match/mismatch with gap in 1

AA-
AGT
AAA

- Sequence 1 with gaps in 2 & 3

AAG
AG-
AA-

- Sequence 2 with gaps in 1 & 3

AA-
AGT
AA-

- Sequence 3 with gaps in 1 & 2

AA-
AG-
AAA

- Choose the largest value among the above seven possibilities

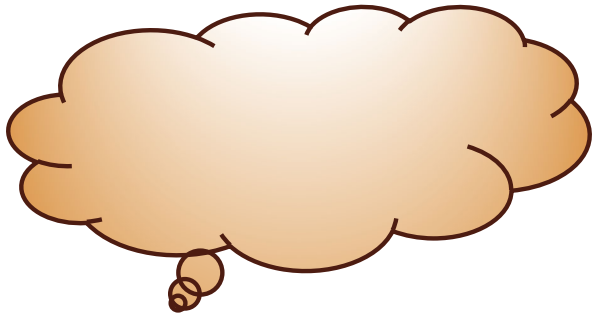
Multiple alignment: dynamic programming

- $s_{i,j,k} = \max$

$$
\begin{cases}
s_{i-1,j-1,k-1} + \delta(v_i, w_j, u_k) & \text{cube diagonal: no indels} \\
s_{i-1,j-1,k} + \delta(v_i, w_j, \_) & \text{face diagonal: one indel} \\
s_{i-1,j,k-1} + \delta(v_i, \_, u_k) & \\
s_{i,j-1,k-1} + \delta(\_, w_j, u_k) & \\
s_{i-1,j,k} + \delta(v_i, \_, \_) & \text{edge diagonal: two indels} \\
s_{i,j-1,k} + \delta(\_, w_j, \_) & \\
s_{i,j,k-1} + \delta(\_, \_, u_k) &
\end{cases}
$$

- $\delta(x, y, z)$ is an entry in the 3D scoring matrix

Challenge



Runtime ????

Computational Complexity

- For protein sequences each 300 amino acid in length & excluding gaps, with DP algorithm
  - Two sequences, 300^2 comparisons
  - Three sequences, 300^3 comparisons
  - N sequences, 300^N comparisons

- The number of comparisons & memory required are too large for n > 3 and not practical

DP-based MSA: running time

- For k sequences of length n, build a k-dimensional Manhattan, with run time $(2^k\text{-}1)(n^k)$; $O(2^k n^k)$

- Conclusion: dynamic programming approach for alignment between two sequences is easily extended to k sequences (simultaneous approach) but it is impractical due to exponential running time.
  - DCA (Divide and Conquer; Stoye et al., 1997), 20-25 sequences
  - OMA (Optimal Multiple Alignment; Reinert et al., 2000)

Heuristics MSA

- Computing exact MSA is computationally almost impossible, and in practice heuristic approaches are used.

- Heuristic approaches to MSA
  I. Greedy alignment
  II. Star alignment
  III. Progressive alignment – ClustalW

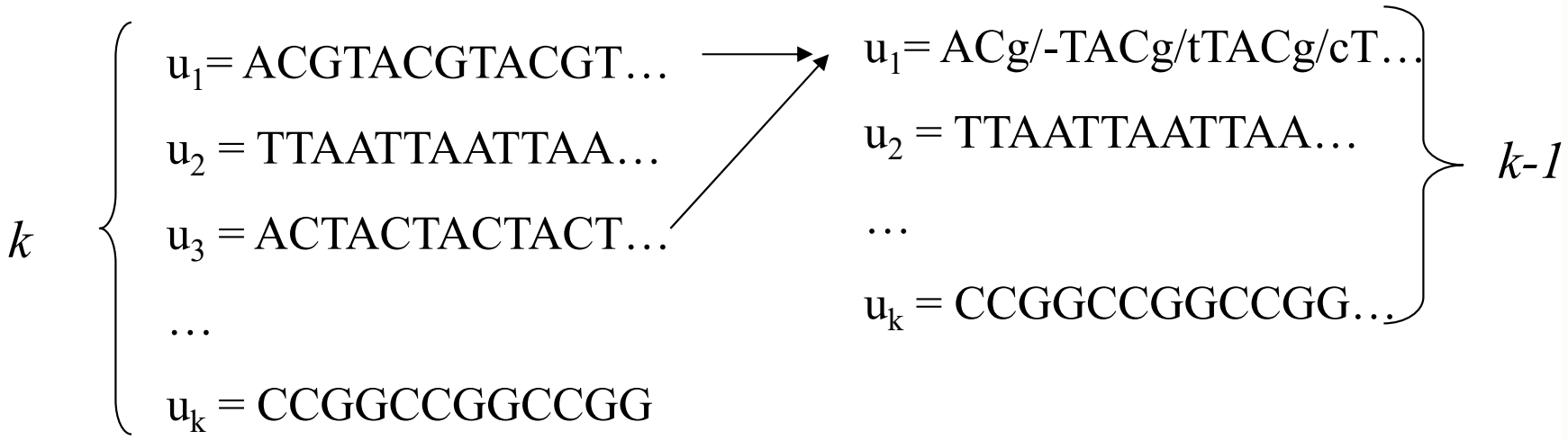
(I) Greedy MSA Algorithm

1. Starts by selecting the two strings having the highest scoring pairwise alignment (among all possible pairs of strings)
2. Uses this pairwise alignment as a building block for iteratively adding one string at a time to the growing multiple alignment.
3. Select the string having maximum score against the current alignment at each stage.

➔ Problem of constructing a multiple alignment of t sequences is reduced to constructing t pairwise alignments

Greedy Approach in General

- Choose most similar pair of strings and combine into a profile, thereby reducing alignment of *k* sequences to an alignment of of *k-1* sequences/profiles. **Repeat.**
- This is a heuristic greedy method.



Greedy Approach: Example

- Consider these 4 sequences

s1	GATTCA
s2	GTCTGA
s3	GATATT
s4	GTCAGC

Greedy Approach: Example (cont'd)

- There are $\binom{4}{2} = 6$ possible alignments.

  score: match = *1* and mismatch/indel = *-1*.

```
s2  GTCTGA                    s1  GATTCA--
s4  GTCAGC (score = 2)        s4  G-T-CAGC (score = 0)

s1  GAT-TCA                   s2  G-TCTGA
s2  G-TCTGA (score = 1)       s3  GATAT-T (score = -1)

s1  GAT-TCA                   s3  GAT-ATT
s3  GATAT-T (score  = 1)      s4  G-TCAGC (score = -1)
```

Greedy Approach: Example (cont'd)

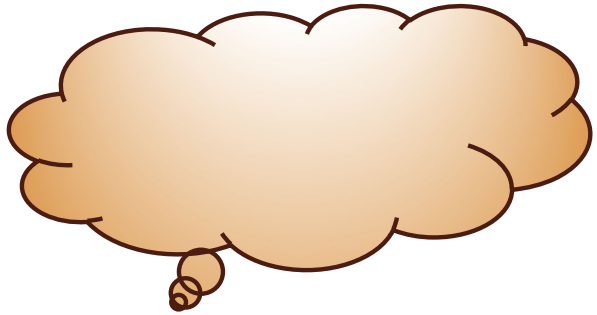
s_2 *and* s_4 are closest; combine:

s2 GTCTGA
s4 GTCAGC
} $s_{2,4}$ (profile) GTCt/aGa/c

new set of 3 sequences:

s_1 GATTCA
s_3 GATATT
$s_{2,4}$ GTCt/aGa/c

Challenge

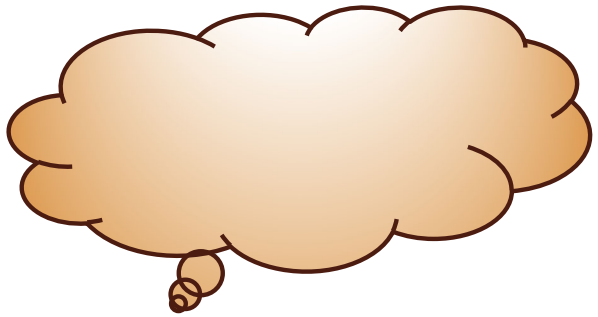


Challenge ????

Runtime ????

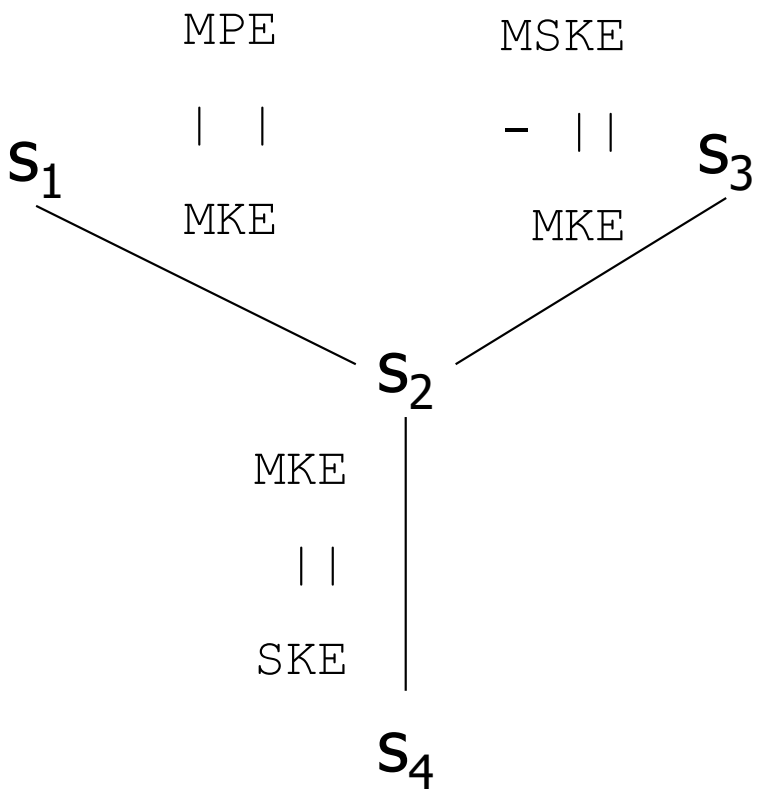
(II) Star Alignments

- Heuristic method for multiple sequence alignments
- Select a sequence s_c as the center of the star
- For each sequence $s_1, \ldots, s_k$ such that index $i \neq c$, perform a global alignment (using DP)
- Aggregate alignments with the principle "once a gap, always a gap"

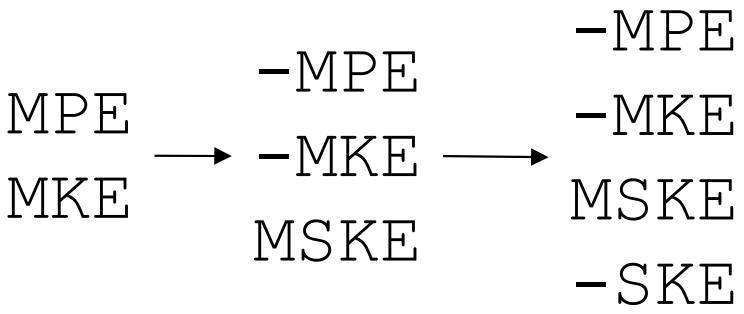


How to choose the center??

Star Alignments Example



s_1: MPE
s_2: MKE
s_3: MSKE
s_4: SKE



Choosing a center

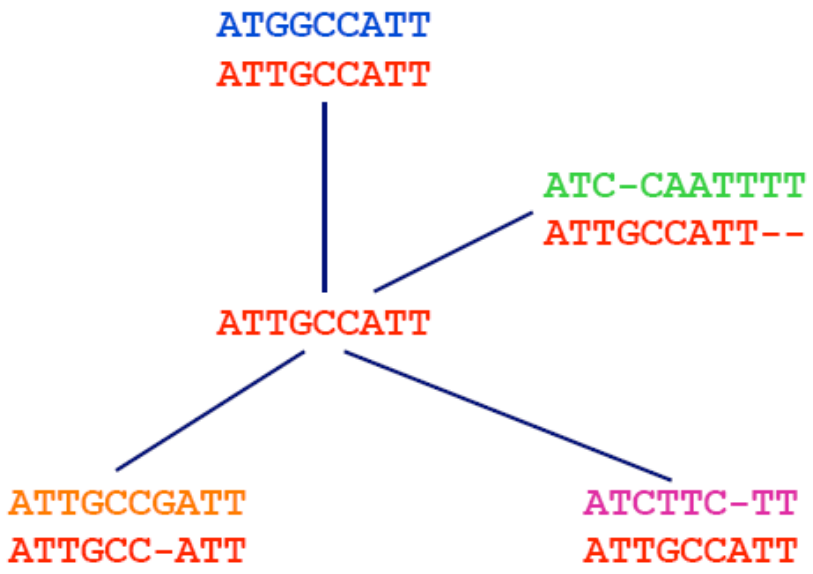
- Try them all and pick the one with the best score
- Calculate all $O(k^2)$ alignments, and pick the sequence s_c that maximizes

$$\sum_{i \neq c} score\,(s_i, s_c)$$

Another Example

- S1=ATTGCCATT
- S2=ATGGCCATT
- S3=ATCCAATTTT
- S4=ATCTTCTT
- S5=ATTGCCGATT

	s_1	s_2	s_3	s_4	s_5	
s_1		7	-2	0	-3	2
s_2	7		-2	0	-4	1
s_3	-2	-2		0	-7	-11
s_4	0	0	0		-3	-3
s_5	-3	-4	-7	-3		-17



Another Example (con't)

Merging Pairwise Alignment

	present pair	alignment
1.	ATGGCCATT ATTGCCATT	ATTGCCATT ATGGCCATT
2.	ATC-CAATTTT ATTGCCATT--	ATTGCCATT-- ATGGCCATT-- ATC-CAATTTT

Another Example (con't)

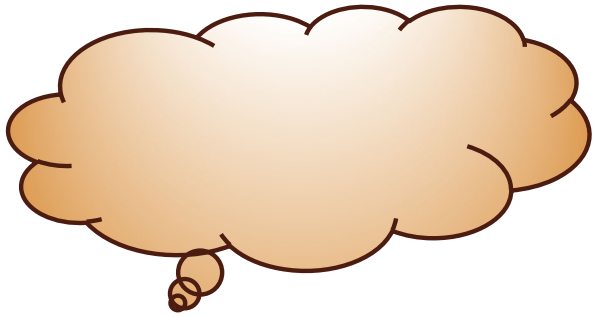
Merging Pairwise Alignment

	present pair	alignment
3.	ATCTTC-TT ATTGCCATT	ATTGCCATT-- ATGGCCATT-- ATC-CAATTTT ATCTTC-TT--
4.	ATTGCCGATT ATTGCC-ATT	ATTGCC-A TT-- ATGGCC-A TT-- ATC-CA-A TTTT ATCTTC- - TT-- ATTGCCG A TT--

shift entire columns
when incorporating a gap

Time Analysis

- Assuming all k sequences have length n
- $O(n^2)$ to calculate global alignment
- $O(k^2)$ global alignments to calculate
- Using a reasonable data structure for joining alignments, no worse than O(kl), where l is upper bound on alignment lengths
- $O(k^2n^2+kl)=O(k^2n^2)$ overall cost



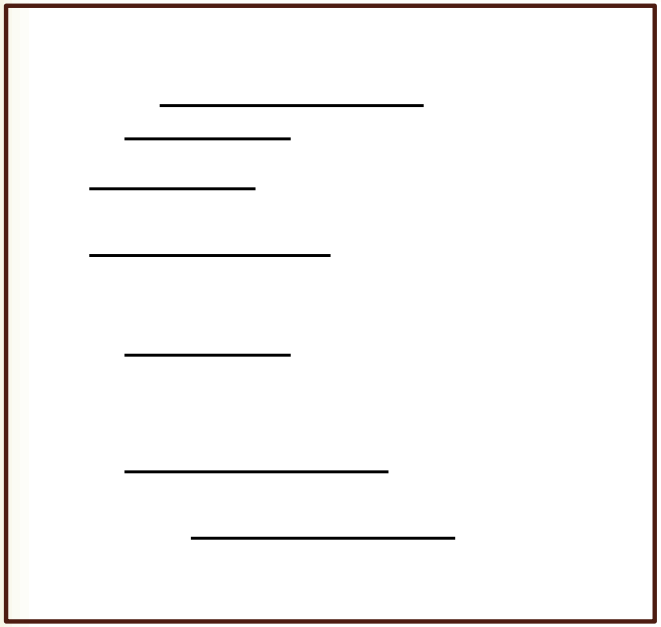
Overall Cost??

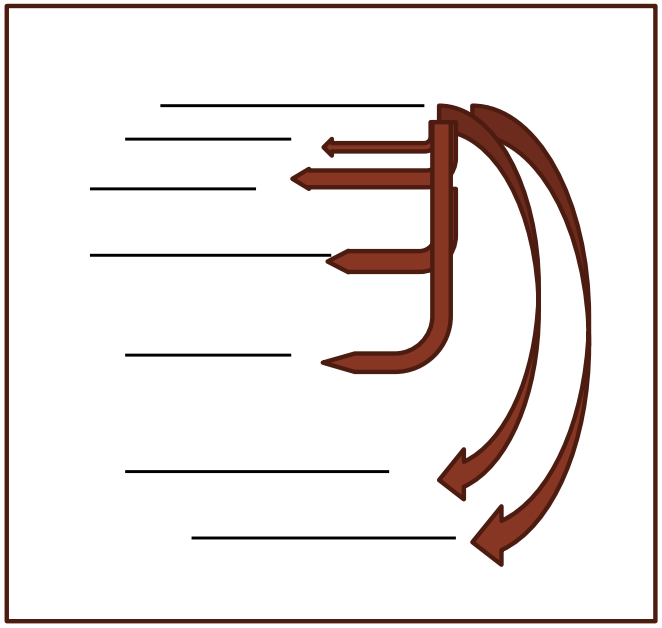
(III) Progressive alignment

- *Progressive alignment* uses guide tree
  - Practical approach for multiple alignment
  - Compare all sequences pair wise
  - Perform cluster analysis
  - Generate a hierarchy for alignment
  - first aligning the most similar pair of sequences
  - Align alignment with next similar alignment or sequence
- Progressive alignment works well for close sequences, but deteriorates for distant sequences

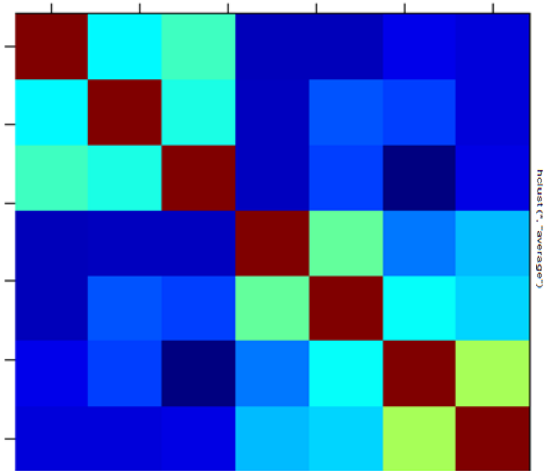
Set of sequences

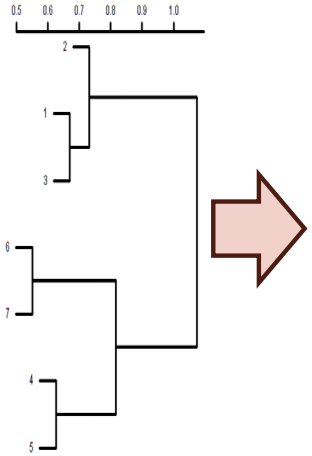
All against all pairwise alignment
Here demonstrated for 1. sequence



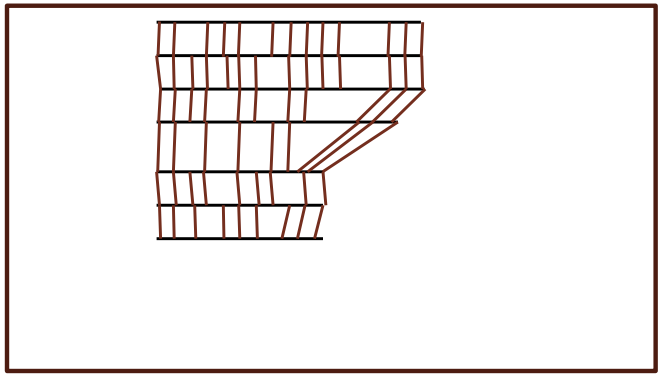


Get pairwise similarities from alignments
Create a cluster tree from similarities





Join sequences in the order obtained
From the cluster tree



ClustalW

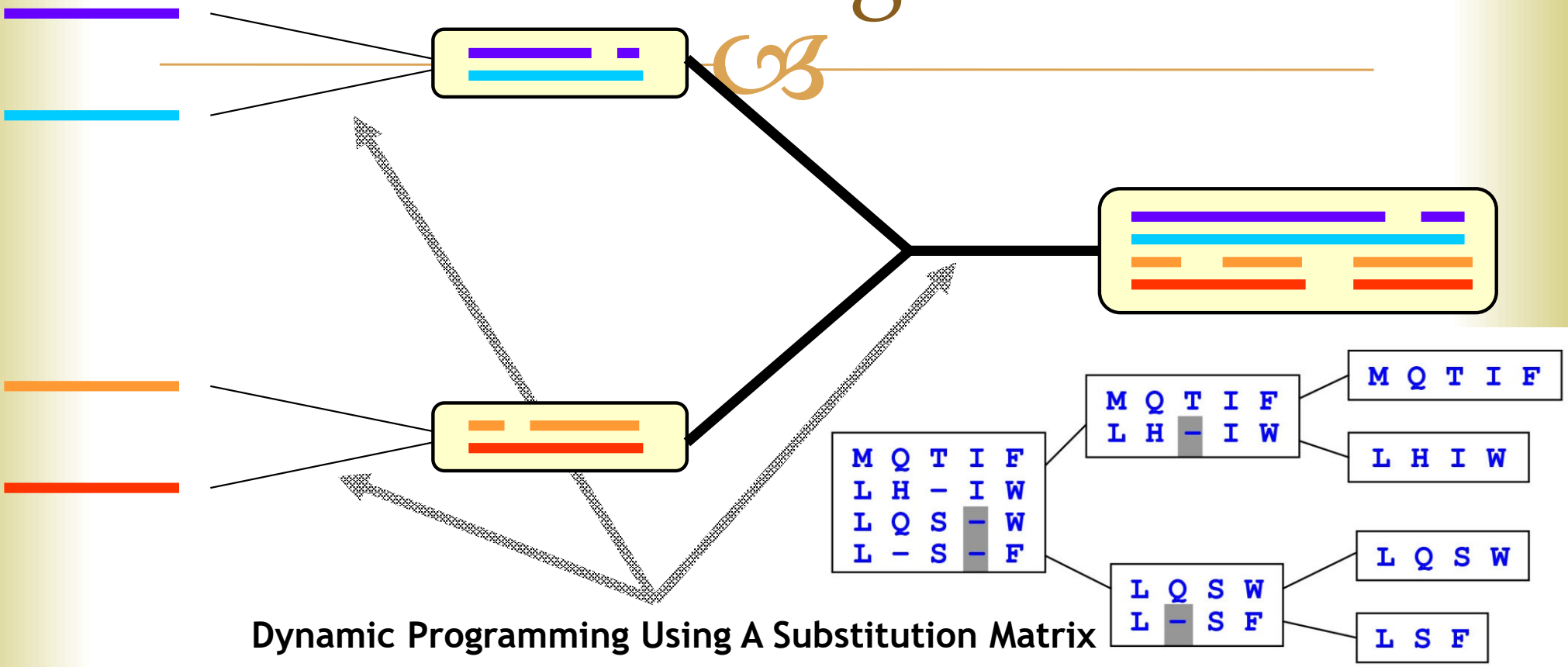
- Popular multiple alignment tool today
- ‘W’ stands for ‘weighted’ (sequences are weighted differently).
- Three-step process

  1.) Construct pairwise alignments

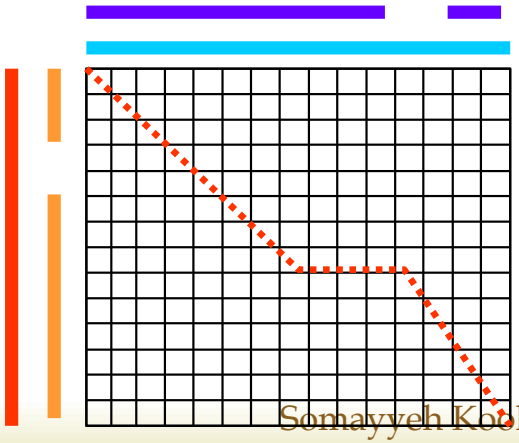
  2.) Build guide tree

  3.) Progressive alignment guided by the tree

ClustalW algorithm



Dynamic Programming Using A Substitution Matrix



Step 1: Pairwise alignment

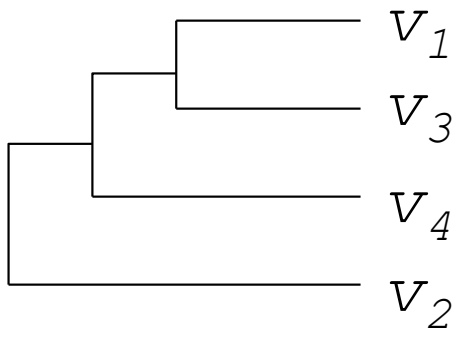
- Aligns each sequence again each other giving a similarity matrix
- Ex. Similarity = exact matches / sequence length (percent identity)

	v_1	v_2	v_3	v_4
v_1	–			
v_2	.17	–		
v_3	.87	.28	–	
v_4	.59	.33	.62	–

(.17 means 17 % identical)

Step 2: Guide tree

	v_1	v_2	v_3	v_4
v_1	-			
v_2	.17	-		
v_3	.87	.28	-	
v_4	.59	.33	.62	-



Calculate:

$v_{1,3}$ = alignment (v_1, v_3)

$v_{1,3,4}$ = alignment$((v_{1,3}), v_4)$

$v_{1,2,3,4}$ = alignment$((v_{1,3,4}), v_2)$

Guide tree *roughly* reflects evolutionary relations

Step 3: Tree based recursion

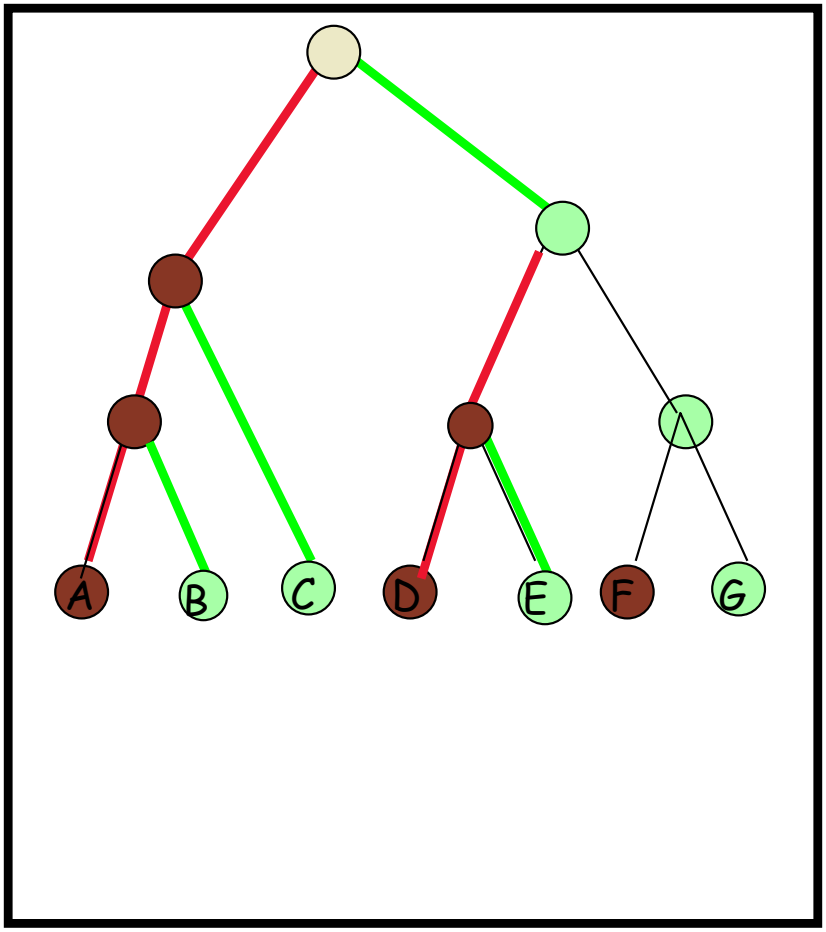
```
Align ( Node N)
        {
        if ( N->left_child is a Node)
                A1=Align ( N->left_child)

        else if ( N->left_child is a Sequence)
                A1=N->left_child

        if (N->right_child is a node)
                A2=Align (N->right_child)

        else if ( N->right_child is a Sequence)
                A2=N->right_child

        Return dp_alignment (A1, A2)
        }
```

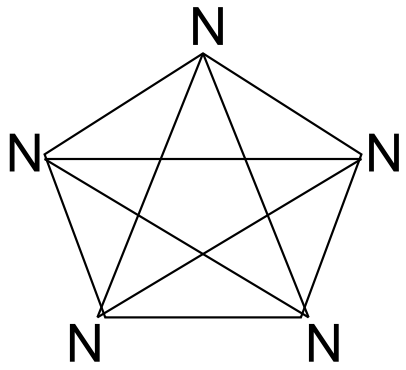


Progressive alignment: Scoring scheme

- Scoring scheme is arguably the most influential component of the progressive algorithm
- Matrix-based algorithms
  - ClustalW, MUSCLE, Kalign
  - Use a substitution matrix to assess the cost of matching two symbols or two profiled columns
    1. Sum of pairs (SP score)
    2. Tree based scoring
    3. Entropy score

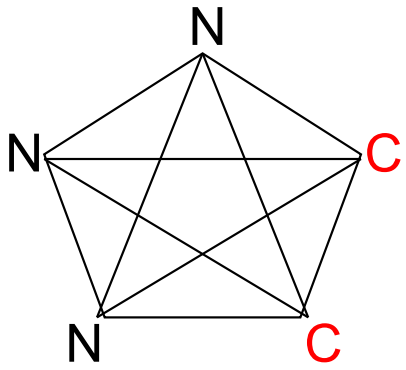
a. Sum of pairs score (SP score)

```
Seq   Column-A -B
1        .....N.............N.....
2        .....N.............N.....
3        .....N.............N.....
4        .....N.............C.....
5        .....N.............C.....
```



Score= 10 * S(N,N)

= 10 * 6 = 60



Score= 3 * S(N,N) + 6 * S(N,C) + S(C,C)

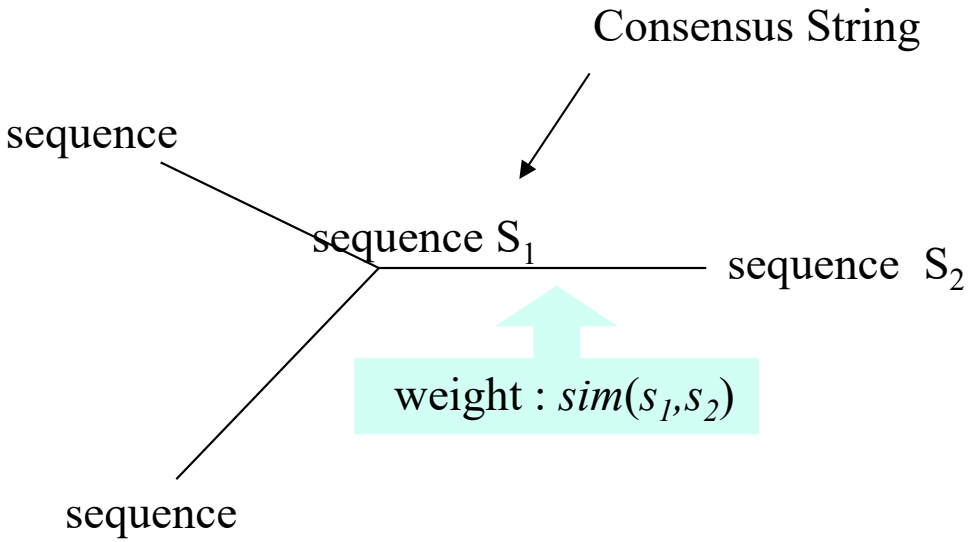
= 3 * 6 + 6 * (-3) + 9 = 9

(BLOSUM62)

Problem ????? **over-estimation of the mutation costs**

b. Tree-based scoring

- Compute the overall similarity based on pairwise alignment along the edge

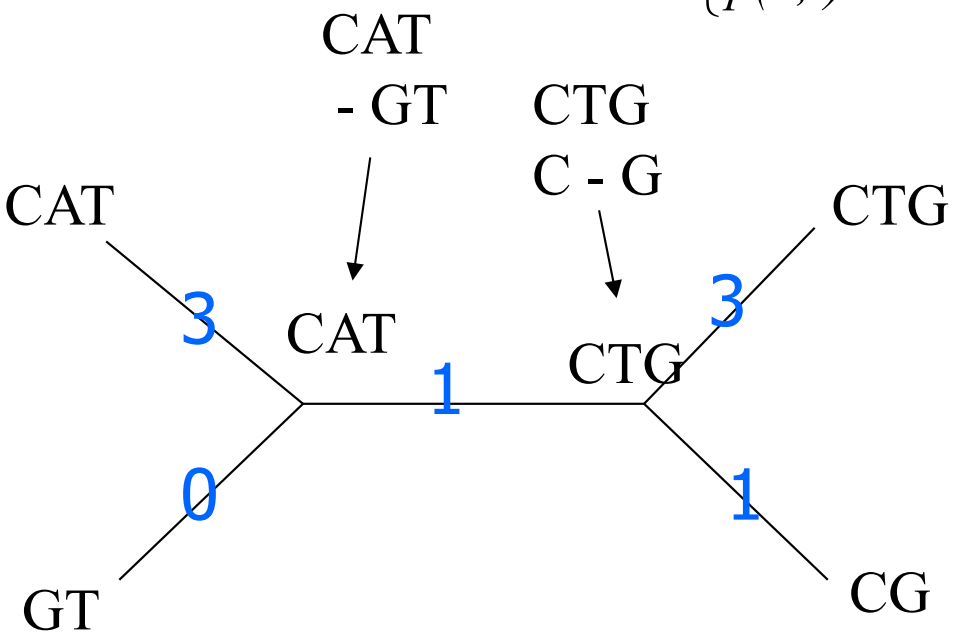


- The sum of all these weights is the *score* of the tree

b. Tree-based scoring

- Scoring system used is

$$\begin{cases} p(a,b) = 1 & if \quad a = b \\ p(a,b) = 0 & if \quad a \neq b \\ p(a,-) = -1 \end{cases}$$



We have a score of 8

c. Entropy-based scoring

In information theory, entropy is a measure of the uncertainty associated with a random variable (a means to quantify information using some kind of currency, usually bits. The rarer, or equivalently more interesting, a thing is, the more bits its worth). The entropy H of a discrete random variable X with possible values $x_1, ..., x_n$ is $H(X) = E(I(X))$, where $I(X)$ is the information content of X.

If p denotes the probability mass function of X then the entropy is,
$H(X) = \sum_i p(x_i)I(x_i) = -\sum_i p(x_i)log_2p(x_i)$.

Assume a genome has the following frequencies in its DNA:
$p(A) = 0.2, p(T) = 0.2, p(C) = 0.3, p(D) = 0.3,$
then its entropy is
$-(0.2log_2(0.2) + 0.2log_2(0.2) + 0.3log_2(0.3) + 0.3log_2(0.3)) = 1.97.$

Entropy: Example

$$entropy\begin{pmatrix} A \\ A \\ A \\ A \end{pmatrix} = 0$$

$$entropy\begin{pmatrix} A \\ T \\ G \\ C \end{pmatrix} = -\sum \frac{1}{4}\log\frac{1}{4} = -4(\frac{1}{4} * -2) = 2$$

Given a DNA sequence, what is its maximum entropy?

Alignment entropy

- Define frequencies for the occurrence of each letter in each column of multiple alignment
  - $p_A = 1$, $p_T=p_G=p_C=0$ (1st column)
  - $p_A = 0.75$, $p_T = 0.25$, $p_G=p_C=0$ (2nd column)
  - $p_A = 0.50$, $p_T = 0.25$, $p_C=0.25$ $p_G=0$ (3rd column)
- Compute entropy of each column

An alignment with 3 columns

A	A	A
A	C	C
A	C	G
A	C	T
0	**0.811**	**2.0**

Alignment entropy= 2.811

Problems with Progressive Algorithms

- Local minimum problem
  - Recall this is a heuristic approach
  - Sequences are added greedily:
    - Multiple alignments are built up from pairwise alignments.
    - The pairwise alignments follow branching in the initial guide tree.
  - No guarantee of a global optimum
  - Any misaligned regions made early on can not be corrected later on.

Problems with Progressive Algorithms

- Sensitivity to alignment parameters
  - Traditional parameters:
    - weight table
    - cost of opening a gap
    - cost of extending a gap
  - Expectation is one set of parameters works well over
    - all sequences in the set
    - all parts of each sequence

Problems with Progressive Algorithms

- Sensitivity to alignment parameters (*cont'd*)
  - A single weight matrix choice will generally work for closely related sequences.
    - weight matrices give highest weight to identities
    - Any weight matrix will work ok if identities dominate
  - For divergent sequences:
    - Nonidentical residues are more significant
    - Scores to these residues are critical
    - Different weight matrices will be required for:
      - different evolutionary distances
      - Different classes of proteins

Problems with Progressive Algorithms

- Sensitivity to alignment parameters *continued*
  - A range of gap penalty values will generally work for closely related sequences.
  - For divergent sequences:
    - The specific choice of gap penalty value becomes critical
    - For proteins gaps don't occur randomly.
      - E.g. Gaps occur between alpha helices and beta strands rather than within them

MSA - Summary

Progress in Progressive Techniques

- Clustal-W (1.8) (Thompson et al., 1994)
  - Automatic substitution matrix
  - Automatic gap penalty adjustment
- T-COFFEE (Notredame et al., 2000)
  - Improvement in Clustal-W by iteration
  - Pair-Wise alignment (Global + Local)
  - Most accurate method but slow
- MAFFT (Katoh et al., 2002)
  - Utilize the FFT for pair-wise alignment
  - Fastest method
  - Accuracy nearly equal to T-COFFEE

References

- http://tcoffee.vital-it.ch/cgi-bin/Tcoffee/tcoffee_cgi/index.cgi
- Recent evolutions of multiple sequence alignment algorithms. 2007, 3(8):e123
- Issues in bioinformatics benchmarking: the case study of multiple sequence alignment. Nucleic Acids Res. 2010 Jul 1
- Chapter 5